



DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: ENGINEERING CAPSTONE PROJECT – 1(23IE4053R/23IE4053A) A.Y. 2026 - 2027

TITLE OF THE PROJECT	AI Codebase Doctor: An AI-Powered Developer Tool for Automated Detection, Explanation, Repair and Verified Improvement of Software Repositories	
S. No.	University ID	Name
1	2420030443	Krishitha Myneni
2	2420030002	Laasya Priya
3	2420030209	Bhanu Prakash
4	2420030574	Sujay
Name of the Guide		Dr. J. Kavitha Reddy

Abstract

Modern software repositories accumulate functional defects, security vulnerabilities, performance bottlenecks and technical debt far faster than teams can review them manually. Conventional static analysis tools such as SonarQube and ESLint rely on fixed rule sets, so they match only surface-level patterns, produce high false positive rates, analyse files in isolation without cross-module context, and stop at detection, leaving the developer to diagnose, correct and verify every issue. The proposed project, AI Codebase Doctor, is an AI-powered developer tool that closes this gap by executing a complete diagnostic and treatment cycle over an entire GitHub repository through the flow of code analysis, AI detection, issue explanation, fix generation, test and benchmark execution, and validation of measured improvement. Implemented as a Python and FastAPI service, the system clones a repository via the GitHub API and parses each source file using Tree-sitter to build language-agnostic Abstract Syntax Trees, call graphs and dependency graphs. These structures, combined with semantically related code retrieved through vector embeddings, are supplied to a Large Language Model that reasons about developer intent rather than matching signatures, enabling detection of logical bugs, security weaknesses mapped to the OWASP Top 10 and CWE taxonomy, performance bottlenecks such as N+1 database queries and avoidable quadratic loops, and architectural decay such as circular dependencies and tight coupling. Every finding is explained with its root cause, impact and exact file and line, after which the model generates a minimal patch that is applied inside a Docker container reproducing the project environment, where the test suite and benchmark harness are executed against a recorded baseline; only patches that pass all tests while improving the target metric are proposed. A representative diagnosis reports an N+1 query in PaperService.java at line 142 costing roughly 420 milliseconds per request, whose optimised replacement yields a validated 38 percent improvement. GitHub Actions triggers the pipeline on every commit and pull request with inline review comments and quality gates, while Jira integration converts confirmed issues into tracked tickets, making technical debt a managed backlog. By uniting artificial intelligence, static analysis, cybersecurity, performance engineering and software architecture in one closed feedback loop, the system replaces subjective, detection-only review with an automated, evidence-based process delivering fewer false positives, shorter review cycles and quantified improvement.

Signature of the Guide

HOD